

C#.NET Properties



Mr. Bangar Raju

NARESH
technologies

Understanding Properties in C#: A Comprehensive Guide

By Albert Harmon

Published December 23, 2016

🕒 4 min read

Education



TLDR This blog post explores the concept of properties in C#, detailing how to define and implement them within a class. It covers various types of properties, including auto-implemented properties and the use of enums for better user

experience. Additionally, it discusses best practices for property accessors and the importance of providing clear options for users.

Welcome to this detailed exploration of properties in C#. This post is a continuation of our previous discussions on properties, where we defined a class called ``customer`` with various attributes. In this part, we will expand on that by adding new attributes and discussing best practices for implementing properties in C#.

Recap of Properties

In the previous parts, we discussed what properties are and how to define them. Properties in C# are special methods that provide a flexible mechanism to read, write, or compute the values of private fields. They are defined using the ``get`` and ``set`` accessors.

Previous Properties Defined

In our ``customer`` class, we initially defined four properties:

- Customer ID
- Status
- Customer Name
- Balance

Adding New Attributes

Today, we will add three more attributes to the `Customer` class:

- City
- State
- Country

Defining the City Property

To start, we will define the `city` property. The type of this property will be a string. Here's how we can implement it:

```
private string _city;
public string City
{
    get { return _city; }
    set
    {
        if (_status == true)
        {
            if (value == "Delhi" || value == "Mumbai" || value
            {
                _city = value;
            }
            else
            {
                throw new ArgumentException("City must be one
```

```
}  
}  
}  
}
```

In this implementation, we check if the status is active before allowing the assignment of a city. Additionally, we ensure that the city is one of the predefined values.

User Experience Considerations

One issue we encountered is that if a user tries to assign a city that is not in our predefined list, they receive no feedback on what values are acceptable. This can lead to frustration. To improve user experience, we should either:

1. Inform the user of the acceptable values.
2. Use an enum to restrict the values that can be assigned.



Using Enums for Better Clarity

Enums are a great way to define a set of named constants. We can define an enum for the cities as follows:

```
public enum Cities  
{
```

```
    Delhi,  
    Mumbai,  
    Chennai,  
    Kolkata,  
    Bengaluru,  
    Hyderabad  
}
```

Now, we can redefine our `city` property to use this enum:

```
private Cities _city;  
public Cities City  
{  
    get { return _city; }  
    set  
    {  
        if (_status == true)  
        {  
            _city = value;  
        }  
    }  
}
```

This way, when a user tries to assign a value to the `city` property, they will have a clear list of acceptable values to choose from.

Defining the State Property

Next, we will define the ``state`` property. For this property, we will keep it simple and allow it to be a string:

```
private string _state;
public string State
{
    get { return _state; }
    set
    {
        if (_status == true)
        {
            _state = value;
        }
    }
}
```

Access Modifiers for Properties

In some cases, we may want to restrict who can set a property. For example, we might want to allow anyone to get the state but only allow child classes to set it. This can be achieved by using access modifiers:

```
public string State
{
    get { return _state; }
```

```
protected set { _state = value; }  
}
```

This way, the `get` accessor is public, while the `set` accessor is protected, allowing only derived classes to modify the state.

Implementing the Country Property

Finally, we will define the `country` property. C# 3.0 introduced auto-implemented properties, which allow us to define properties without explicitly declaring a backing field:

```
public string Country { get; set; }
```

This simplifies property definitions significantly. However, if we need to enforce any conditions, we cannot use auto-implemented properties.

Conclusion

In this session, we have defined seven properties in total for our `Customer` class. Properties play a crucial role in encapsulating the attributes of a class, providing controlled access to the data. By using enums and access modifiers effectively, we can enhance user experience and maintain data integrity.

Thank you for following along in this tutorial. For more insights and tutorials, stay tuned for our next video and subscribe to our channel.